

Sistema Hospitalario: Evolución de un Paradigma Estructurado a Programación Orientada a Objetos

Informe Final — Versiones 2 y 2.1

Rodrigo André Quesada Mamani (45%)

Diego Mauricio Champi Pareja (35%)

Alexander Quispe Ccapa (8%)

Duvan Rodrigo Huaccanqui Velasquez (8%)

Nilton Peña Fernández (4%)

Docente: Ing. Julio Cesar Carbajal Luna

Asignatura: Estructuras de Datos y Algoritmos I

2025

Resumen

El presente informe técnico documenta la evolución del Sistema Hospitalario, un proyecto de software desarrollado en el contexto de la asignatura Estructuras de Datos y Algoritmos I. El documento describe la transición desde una arquitectura inicial basada en programación estructurada (V1) hacia un diseño moderno empleando el paradigma de Programación Orientada a Objetos (POO) en sus versiones 2 y 2.1. Se detallan las decisiones de diseño que fundamentaron la migración, incluyendo la identificación de problemas de acoplamiento en el código original, la implementación de patrones de diseño orientada a objetos, y la incorporación de mecanismos robustos de persistencia y manejo de excepciones. La versión 2 introduce los conceptos fundamentales de encapsulamiento, herencia y polimorfismo mediante el lenguaje C++, mientras que la versión 2.1 extiende estas capacidades con persistencia en archivos CSV y técnicas avanzadas de gestión de recursos basadas en el principio RAIL (Resource Acquisition Is Initialization). El análisis comparativo demuestra una mejora significativa en la mantenibilidad, extensibilidad y robustez del código resultante.

Palabras clave: Programación Orientada a Objetos, C++, Herencia, Polimorfismo, RAIL, Persistencia CSV, Refactorización, Encapsulamiento

Sección 1: Instrucciones de Compilación y Ejecución del Proyecto

1.1 Requisitos del Compilador

El Sistema Hospitalario V2 y V2.1 está desarrollado en C++14/C++17. A continuación se detallan los requisitos específicos del compilador:

Plataforma	Compilador Recomendado	Versión Mínima
Windows	MinGW-w64 (GCC)	GCC 8.1.0+
Linux/macOS	GCC (g++)	GCC 7.0.0+
Windows (alternativo)	MSVC (Visual Studio)	VS 2019+
Todas las plataformas	Clang++	Clang 6.0+

1.2 Verificación de la Versión del Compilador

Antes de compilar el proyecto, es fundamental verificar que el compilador instalado cumpla con los requisitos mínimos. Ejecute los siguientes comandos según su sistema operativo:

Windows (MinGW-w64):

```
$ g++ --version
```

Linux/macOS:

```
$ g++ --version
```

Clang:

```
$ clang++ --version
```

La salida esperada debe mostrar un número de versión igual o superior a los especificados en la tabla anterior. Si la versión es inferior, proceda a la sección de instalación.

1.3 Guía de Instalación del Compilador

1.3.1 Instalación en Windows (MinGW-w64)

1. Descargue una versión reciente de **WinLibs** (GCC para Windows) desde:
<https://winlibs.com/>
2. Extraiga el archivo comprimido en una ubicación permanente, por ejemplo:
`C:\WinLibs`
3. Verifique que la carpeta bin del compilador exista, por ejemplo:
`C:\WinLibs\mingw64\bin`
4. Abra **Embarcadero Dev-C++** y diríjase a:
Herramientas → Opciones del Compilador → Programas
5. Configure la ruta del compilador (g++.exe) apuntando al ejecutable ubicado dentro de la carpeta bin de WinLibs. Si Dev-C++ no detecta automáticamente el compilador, seleccione manualmente la ruta correspondiente.
6. Guarde los cambios y reinicie Dev-C++ para aplicar la configuración.
7. Verifique la instalación abriendo una terminal (CMD o PowerShell) y ejecutando:
`g++ --version`

Si se muestra la versión del compilador GCC, la instalación se realizó correctamente.

Nota: Se recomienda utilizar WinLibs junto con Embarcadero Dev-C++ porque proporciona un entorno de compilación sencillo y evita los problemas de configuración de variables de entorno y múltiples entornos de ejecución que pueden presentarse con otras alternativas.

1.3.2 Instalación en Linux (Ubuntu/Debian)

1. Abra una terminal.
2. Actualice los repositorios: `sudo apt update`

3. Instale GCC: `sudo apt install build-essential`
4. Verifique la versión: `g++ --version`

1.3.3 Instalación en macOS

1. Instale Homebrew si no lo tiene: `/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"`
2. Instale GCC: `brew install gcc`
3. Verifique la versión: `g++-13 --version` (o la versión instalada)

1.4 Compilación y Ejecución del Proyecto

Una vez verificada la instalación del compilador, proceda con los siguientes pasos:

1.4.1 Versión 2 (POO básica)

Navegue al directorio `v2/` del proyecto y ejecute el siguiente comando de compilación:

```
g++ -std=c++17 -Wall -Wextra -o sistema_hospitalario main.cpp
Persona.cpp Paciente.cpp Personal.cpp Cita.cpp GestorPacientes.cpp
GestorCitas.cpp SistemaHospitalario.cpp
```

Para ejecutar el programa compilado:

```
./sistema_hospitalario
```

1.4.2 Versión 2.1 (Con persistencia CSV)

Navegue al directorio `v2.1/` del proyecto:

```
g++ -std=c++14 -Wall -Wextra -o sistema_hospitalario main.cpp
Menu.cpp Persona.cpp Paciente.cpp Personal.cpp Cita.cpp
GestorPacientes.cpp GestorPersonal.cpp GestorCitas.cpp Persistencia.cpp
SistemaHospitalario.cpp
```

Ejecución:

```
./sistema_hospitalario
```

La versión 2.1 generará automáticamente los archivos CSV de datos de prueba en el directorio datos/ si estos no existen.

Sección 2: Introducción

El desarrollo de software hospitalario presenta desafíos significativos en términos de complejidad lógica, integridad de datos y escalabilidad del sistema. En el contexto de la asignatura Estructuras de Datos y Algoritmos I, se desarrolló un Sistema Hospitalario cuya primera versión (V1) implementó una arquitectura basada en programación estructurada. Aunque esta aproximación inicial permitió establecer las funcionalidades básicas del sistema, evidenció limitaciones inherentes al paradigma que comprometían la calidad del código a largo plazo.

La programación estructurada, si bien constituye la base fundamental de la ingeniería de software, presenta características que dificultan la evolución de sistemas de mediana a gran escala. La dispersión de datos y comportamientos relacionados a través de múltiples módulos genera lo que se conoce como bajo acoplamiento de datos, donde la información relevante se encuentra fragmentada y sin una cohesión lógica clara. En sistemas como los hospitalarios, donde la complejidad de las entidades (pacientes, personal médico, citas, historial clínico) demanda una representación fiel de las relaciones del mundo real, esta fragmentación se traduce en código difícil de mantener, extender y depurar.

El presente informe documenta el proceso de migración desde esta arquitectura estructurada hacia un diseño basado en el paradigma de Programación Orientada a Objetos (POO), materializado en las versiones 2 y 2.1 del Sistema Hospitalario. Esta transición no responde únicamente a imperativos académicos, sino que aborda problemas concretos identificados en la versión original: la proliferación de variables globales para compartir estado entre funciones, la ausencia de mecanismos de validación centralizados, y la dificultad para representar las

relaciones jerárquicas entre entidades del dominio (por ejemplo, la relación entre personas y sus especializaciones como pacientes o personal médico).

La decisión de adoptar POO se fundamenta en los principios establecidos por autores como Stroustrup (2014), quien destaca que "el diseño orientado a objetos proporciona al programador medios para organizar el programa de manera que refleje la estructura del problema que resuelve" (p. 21). Similarly, Gamma et al. (1995) establecen que los patrones de diseño orientados a objetos "facilitan la reutilización y extensibilidad del software" (p. 3). En las siguientes secciones, se detallará cómo estos principios se aplican sistemáticamente en la implementación del sistema, desde la definición de clases con encapsulamiento apropiado hasta el uso de herencia y polimorfismo para lograr un código más mantenible y robusto.

Sección 3: Fase de Refactorización — Versión 2

La versión 2 del Sistema Hospitalario representa una reimplementación completa del sistema, adoptando los principios fundamentales de la Programación Orientada a Objetos. Esta fase de refactorización se centró en tres objetivos principales: la agrupación coherente de estados y comportamientos en clases bien definidas, la implementación de mecanismos de protección de datos mediante encapsulamiento, y el establecimiento de una jerarquía de clases que reflejara las relaciones del dominio hospitalario.

3.1 Agrupación de Estados y Comportamientos en Clases

La transición de programación estructurada a POO requirió identificar las entidades fundamentales del dominio hospitalario y representar cada una como una clase cohesiva. El análisis del dominio reveló tres entidades principales: Paciente, Personal y Cita, cada una con sus propios atributos de estado y comportamientos asociados.

La clase Paciente encapsula todos los datos relevantes de un paciente hospitalario: identificador (DNI), nombre, tipo de sangre, fecha de nacimiento, dirección, información de contacto, contacto de emergencia, historial de alergias y un vector de entradas de historial médico. Los comportamientos incluyen validaciones (`validarDni`, `validarTipoSangre`), gestión de alergias (`agregarAlergia`, `removerAlergia`, `tieneAlergia`) y gestión del historial clínico.

La clase Personal representa al personal médico y administrativo del hospital. Hereda los atributos básicos de Persona (`id`, nombre, teléfono) y añade características específicas como rol (`director`, `especialista`, `médico general`, `enfermera`), especialidad, turno de trabajo, nivel de acceso y conjunto de permisos. Los comportamientos incluyen validación de roles y turnos, control de

acceso basado en permisos, y métodos para verificar jerarquía entre personal.

La clase Cita modela las citas médicas, con atributos como identificador de cita, DNI del paciente, ID del médico, fecha, estado (pendiente, en atención, completada, cancelada), motivo, notas y lista de medicación actual. Los comportamientos permiten modificar estados, agregar o remover medicamentos, y validar estados válidos.

3.2 Implementación de Encapsulamiento

El encapsulamiento constituye el primer pilar de la POO implementado en el Sistema Hospitalario V2. Meyers (2014) enfatiza que "el encapsulamiento mejora la mantenibilidad al limitar el impacto de los cambios en la implementación interna" (p. 38). En nuestro sistema, esta filosofía se materializa mediante la protección rigurosa de los atributos de clase.

En la clase Persona (superclase), los atributos `id_`, `nombre_` y `telefono_` se declaran como `protected`, permitiendo el acceso desde las subclases Paciente y Personal mientras se oculta la implementación a código externo. Las clases derivadas utilizan estos atributos directamente, pero cualquier acceso desde el exterior debe realizarse a través de getters públicos.

En las clases Paciente y Personal, todos los atributos son privados (`private`), garantizando que ningún código externo pueda modificar el estado del objeto sin pasar por los setters validados. Por ejemplo, el método `setEmail()` en Paciente valida que el correo electrónico contenga el carácter '@' antes de realizar la asignación:

```
bool Paciente::setEmail(const std::string& email) {
```

```

    if (email.find('@') == std::string::npos) return false;

    email_ = email;

    return true;

}

```

Esta validación centralizada en los setters asegura que el estado del objeto siempre sea coherente e invariable, un principio fundamental del encapsulamiento descrito por Joyanes Aguilar (2020): "el encapsulamiento permite ocultar la información, de manera que sólo se puede acceder a los datos a través de los métodos públicos" (p. 312).

3.3 Constructores con Listas de Inicialización

Los constructores del sistema implementan listas de inicialización, una práctica recomendada en C++ que mejora tanto el rendimiento como la seguridad del código. La lista de inicialización asigna valores a los atributos directamente durante la construcción del objeto, evitando la ejecución de constructores por defecto seguida de asignaciones.

Un ejemplo característico se encuentra en la clase Paciente:

```

Paciente::Paciente(const std::string& dni, const std::string& nombre,
                  const std::string& tipoSangre, const std::string& fechaNacimiento,
                  const std::string& telefono, const std::string& direccion,
                  const std::string& email, const std::string& contactoEmergencia)
: Persona(dni, nombre, telefono),
  tipoSangre_(tipoSangre),

```

```
    fechaNacimiento_(fechaNacimiento),  
    direccion_(direccion),  
    email_(email),  
    contactoEmergencia_(contactoEmergencia),  
    alergias_(),  
    historial_()  
    { }
```

Stroustrup (2014) explica que "la lista de inicialización es la forma más eficiente de inicializar los miembros de una clase" (p. 297), especialmente importante para tiposconst y referencias que no pueden ser asignados después de la construcción. Además, esta aproximación garantiza que el objeto entre en un estado válido desde el momento de su creación.

3.4 Herencia Simple

La herenciaimple constituye el segundo pilar de la POO implementado. La clase abstracta Persona actúa como base para las clases Paciente y Personal, estableciendo una jerarquía que refleja el dominio del problema. Esta relación se declara formalmente como:

```
class Paciente : public Persona { ... };  
  
class Personal : public Persona { ... };
```

La clase Persona define atributos protected (id_, nombre_, telefono_) que las subclases heredan y acceden directamente, evitando la duplicación de código. Además, Persona declara métodos virtuales puros (mostrarInfo() y getTipo()) que las subclases deben implementar, garantizando

una interfaz común para el procesamiento polimórfico.

Joyanes Aguilar (2020) define la herencia como "la propiedad por la cual una clase permite heredar las características (atributos y métodos) de otra clase" (p. 341). En el contexto hospitalario, tanto pacientes como personal son personas con atributos comunes, pero difieren en sus roles, permisos y comportamientos específicos. La herencia captura precisamente esta relación, permitiendo modelar la realidad del dominio con fidelidad.

3.5 Polimorfismo y Enlace Dinámico

El polimorfismo, tercer pilar de la POO implementado, se manifiesta en el sistema mediante dos mecanismos complementarios: métodos virtuales con override y enlace dinámico a través de punteros de la clase base.

La clase Persona declara métodos virtuales puros:

```
virtual void mostrarInfo() const = 0;  
  
virtual std::string getTipo() const = 0;
```

Estas declaraciones obligan a las subclases Paciente y Personal a proporcionar implementaciones concretas, marcadas explícitamente con la palabra clave override. El sistema puede entonces almacenar punteros Persona* que referencian objetos de cualquier subclase:

```
std::vector<Persona*> registroGeneral_  
  
Persona* p1 = new Paciente(...);
```

```
Persona* p2 = new Personal(...);
```

Al invocar un método virtual a través de un puntero de la clase base, la resolución del método concreto se realiza en tiempo de ejecución mediante la tabla virtual (vtable). Este comportamiento, denominado enlace dinámico o late binding, permite que el siguiente código procese uniformemente objetos de tipos diferentes:

```
for (Persona* p : registroGeneral_) {  
    p->mostrarInfo(); // Invoca Paciente::mostrarInfo() o Personal::mostrarInfo()  
}
```

Gamma et al. (1995) describen el polimorfismo como "la posibilidad de tratar de manera uniforme a objetos de distintas clases" (p. 83), y destacan que esta capacidad es fundamental para escribir código extensible. El destructor virtual en Persona es igualmente crucial: al eliminar un Persona* que apunta a un Paciente, el destructor virtual garantiza que se invoque ~Paciente() antes de ~Persona(), evitando fugas de memoria.

Sección 4: Persistencia y Robustez — Versión 2.1

La versión 2.1 extiende la versión 2 con capacidades de persistencia y robustez, abordando limitaciones críticas para un sistema de producción. El análisis de la versión anterior identificó dos deficiencias principales: la ausencia de mecanismos para conservar datos entre ejecuciones y la falta de manejo estructurado de errores, que provocaba colapsos del sistema ante situaciones inesperadas.

4.1 Decisión de Diseño: Remoción de Pilas y Colas

La versión 2 incluía estructuras de datos lineales (pilas y colas) para gestionar la lista de espera de pacientes y operaciones de deshacer para la medicación. Aunque estas estructuras son fundamentales en el estudio de estructuras de datos, su inclusión en el código de producción presentaba problemas prácticos:

La lista de espera (implementada como cola FIFO) y la pila de deshacer para medicación (implementada como pila LIFO) incrementaban la complejidad del modelo de datos sin aportar valor significativo para la funcionalidad core del sistema hospitalario. En un escenario real, la gestión de espera de pacientes requiere algoritmos sofisticados de priorización que una simple cola no puede proporcionar, y la reversión de medicación tiene implicaciones médicas que requieren validación profesional, no un simple pop de pila.

Adicionalmente, estas estructuras complicaban la serialización a CSV, ya que requerían representar estados transitorios (elementos en espera, historial de deshacer) que no correspondían a datos persistentes del sistema. La decisión de removerlas en la versión 2.1 simplificó

significativamente el modelo de datos, facilitando tanto la persistencia como el mantenimiento del código.

4.2 Persistencia en Archivos CSV

La persistencia en la versión 2.1 se implementa mediante archivos CSV (Comma-Separated Values), aprovechando la biblioteca estándar `<fstream>` de C++. Esta decisión de diseño equilibra simplicidad, interoperabilidad y eficiencia para el volumen de datos de un sistema hospitalario académico.

La clase `Persistencia` encapsula toda la lógica de entrada/salida, proporcionando métodos para cargar y guardar las tres entidades principales: pacientes, personal y citas. El formato CSV chosen prioriza la legibilidad humana y la facilidad de edición manual, características valiosas durante el desarrollo y pruebas.

Ejemplo del formato `pacientes.csv`:

```
dni,nombre,tipoSangre,fechaNacimiento,telefono,direccion,email,contactoEmergencia,alergias
12345678,Maria Garcia Lopez,O+,1990-05-15,987654321,Av. Lima
123,maria@email.com,Juan Garcia,penicilina;aspirina
```

La clase `Persistencia` incluye generación automática de datos de prueba (mock data) en el directorio `datos/`, permitiendo que el sistema funcione inmediatamente después de la compilación sin necesidad de configuración manual. Este mecanismo `plug-and-play` es especialmente valioso para demostraciones y pruebas.

4.3 Técnica RAII para Gestión de Flujos

La versión 2.1 implementa el principio RAI (Resource Acquisition Is Initialization) para la gestión automática de flujos de archivo. Meyers (2014) describe RAI como "una técnica que vincula la gestión de recursos a la duración de los objetos" (p. 196), garantizando que los recursos se liberen correctamente incluso ante excepciones.

En el contexto de flujos de archivo, RAI se manifiesta de manera elegante: los objetos `std::ifstream` y `std::ofstream` son declarados como variables locales dentro de cada método de Persistencia. El destructor de estos objetos se invoca automáticamente al salir del scope, cerrando el archivo sin necesidad de llamadas explícitas a `close()`.

```
void Persistencia::cargarPacientes(GestorPacientes& gestor) {  
    std::ifstream archivo(rutaPacientes_); // Archivo se abre aquí  
  
    if (!archivo.is_open()) {  
        throw std::runtime_error("No se pudo abrir...");  
    }  
  
    std::string linea;  
    while (std::getline(archivo, linea)) {  
        // Procesamiento...  
    }  
} // ~ifstream() se invoca aquí automáticamente, cerrando archivo
```

Esta aproximación presenta dos ventajas críticas:

Primero, elimina las fugas de recursos: si una excepción se lanza durante el procesamiento del archivo, el destructor del ifstream se ejecuta durante el unwinding de la pila, garantizando el cierre del archivo.

Segundo, reduce la complejidad del código: no es necesario mantener tracking manual del estado del archivo ni asegurar que close() se invoque en todos los caminos de ejecución (incluyendo ramas de error).

4.4 Manejo de Errores mediante Excepciones

El sistema implementa una estrategia robusta de manejo de excepciones que previene el colapso del programa ante errores recuperables. Stroustrup (2014) argumenta que "las excepciones proporcionan una forma de transferir control a un handler cuando algo sale mal" (p. 371), y destaca que el manejo adecuado de errores es fundamental para software confiable.

La jerarquía de excepciones del sistema incluye dos tipos principales:

std::runtime_error: Se lanza cuando falla una operación de archivo (apertura, escritura). Ejemplo:

```
throw std::runtime_error(  
    "Error crítico: No se pudo crear el archivo '" + rutaPacientes_  
    + "'. Verifique permisos del directorio.");
```

std::invalid_argument: Se lanza cuando los datos CSV están corruptos o incompletos:

```
throw std::invalid_argument(  
    "Datos corruptos en " + rutaPacientes_ + " línea "  
    + std::to_string(lineaNum) + ": se esperaban al menos 8 campos...");
```

La captura de excepciones se realiza en múltiples niveles del sistema:

Nivel de persistencia: Los métodos de la clase Persistencia delegan la responsabilidad de validar datos al código que los llama, lanzando excepciones solo para errores de infraestructura (archivo no accesible).

Nivel de sistema: SistemaHospitalario envuelve las llamadas a persistencia en bloques try-catch que capturan excepciones específicas.

Nivel de interfaz: Menu captura std::exception& genérica para cada operación del menú, imprimiendo el mensaje de error y permitiendo que el usuario continúe operando el sistema sin reiniciar.

Esta estrategia de múltiples niveles, combinada con la técnica RAII, asegura que el sistema nunca colapse abruptamente. Ante un error, el usuario recibe información contextualizada sobre el problema y puede tomar acciones correctivas.

Sección 5: Aplicación de Conceptos de POO

Esta sección proporciona una referencia consolidada que indica explícitamente dónde se aplican los principales conceptos de Programación Orientada a Objetos en el código del Sistema Hospitalario V2 y V2.1.

Concepto	Ubicación en el Código	Descripción
Herencia	Persona.h (líneas 15-35) Paciente.h (línea 23) Personal.h (línea 19)	Clase abstracta Persona como base; Paciente y Personal heredan de ella. Atributos protected para acceso desde subclases.
Polimorfismo	Persona.h (líneas 37-38, virtuales puros) SistemaHospitalario.cpp (líneas 45-70)	Métodos virtuales puros en Persona; override en Paciente/Personal. vector<Persona*> procesa objetos de distintas subclases.
Downcasting (RTTI)	SistemaHospitalario.cpp (líneas 72-100) Paciente.h (líneas 58-60) Personal.h (líneas 64-66)	dynamic_cast<Paciente*>(persona) para invocar métodos exclusivos. mostrarHistorialCompleto() y mostrarPermisos().
Gestión de Errores	Persistencia.cpp	throw std::runtime_error,

	(múltiples puntos) SistemaHospitalario.cpp (bloques try-catch) Menu.cpp (captura genérica)	std::invalid_argument. Captura multinivel para evitar colapsos del sistema.
Persistencia (RAII)	Persistencia.h (declaraciones) Persistencia.cpp (implementación)	std::ifstream/ofstream como variables locales. destructores cierran automáticamente archivos (RAII).

5.1 Herencia — Detalle de Implementación

La jerarquía de herencia se establece en Persona.h con la declaración:

```
class Persona {
protected:
    std::string id_;
    std::string nombre_;
    std::string telefono_;
public:
    virtual ~Persona();
    virtual void mostrarInfo() const = 0;
    virtual std::string getTipo() const = 0;
};
```

La herencia simple se implementa con especificador public en las clases derivadas:

```
class Paciente : public Persona { ... };  
  
class Personal : public Persona { ... };
```

Este diseño permite que Paciente reutilice directamente id_, nombre_ y telefono_ sin necesidad de redeclararlos, mientras que Personal añade sus propios atributos especializados (rol_, especialidad_, permisos_).

5.2 Polimorfismo — Procesamiento Uniforme

El polimorfismo permite procesar una colección heterogénea de objetos mediante una interfaz común. El método listarRegistroGeneral() en SistemaHospitalario demuestra este principio:

```
void SistemaHospitalario::listarRegistroGeneral() const {  
    for (size_t i = 0; i < registroGeneral_.size(); ++i) {  
        registroGeneral_[i]->mostrarInfo();  
    }  
}
```

El vector registroGeneral_ almacena punteros Persona*, pero los objetos reales son instancias de Paciente o Personal. La llamada a mostrarInfo() se resuelve dinámicamente: para Paciente se ejecuta Paciente::mostrarInfo() (que muestra tipo de sangre, alergias, historial), mientras que para Personal se ejecuta Personal::mostrarInfo() (que muestra rol, especialidad, permisos).

5.3 Downcasting — Acceso a Métodos Exclusivos

El downcasting permite acceder a métodos específicos de las subclases cuando se trabaja con punteros de la clase base. El sistema utiliza `dynamic_cast<>` para verificar el tipo real del objeto en tiempo de ejecución:

```
void SistemaHospitalario::identificarYProcesar(Persona* persona) {  
  
    Paciente* paciente = dynamic_cast<Paciente*>(persona);  
  
    if (paciente != nullptr) {  
  
        paciente->mostrarHistorialCompleto(); // Método exclusivo de Paciente  
  
        return;  
  
    }  
  
    Personal* personal = dynamic_cast<Personal*>(persona);  
  
    if (personal != nullptr) {  
  
        personal->mostrarPermisos(); // Método exclusivo de Personal  
  
        return;  
  
    }  
  
}
```

El operador `dynamic_cast` devuelve `nullptr` si la conversión no es válida (tipo incompatible), permitiendo verificar la seguridad de la operación antes de invocaciones que de otro modo serían inválidas.

Sección 6: Conclusiones

El análisis comparativo entre la versión base (V1, programación estructurada) y las versiones 2 y 2.1 (POO) del Sistema Hospitalario evidencia mejoras sustanciales en múltiples dimensiones de calidad del software. Esta sección documenta los hallazgos principales del análisis de mantenibilidad.

Dimensión de Cohesión y Acoplamiento

La versión estructurada original presentaba un bajo acoplamiento de datos, donde atributos relacionados (como el DNI y el historial de alergias de un paciente) se encontraban dispersos en múltiples estructuras de datos globales. La transición a POO agrupó estos elementos en clases cohesivas, donde Paciente encapsula todo lo relacionado con pacientes, Personal todo lo relacionado con el staff médico, y Cita todo lo relacionado con citas médicas.

Esta reorganización reduce drásticamente el acoplamiento entre módulos: cada gestor (GestorPacientes, GestorPersonal, GestorCitas) manipula exclusivamente objetos de su tipo correspondiente, eliminando dependencias indirectas que existían en la versión estructurada. Según Gamma et al. (1995), "un diseño con buena cohesión de componentes y bajo acoplamiento entre ellos es más fácil de desarrollar, mantener y testar" (p. 82).

Extensibilidad del Código

La arquitectura POO facilita la extensión del sistema sin modificar código existente, aplicando el

Principio de Abierto/Cerrado (Open/Closed Principle): las clases están abiertas para extensión pero cerradas para modificación. Para añadir un nuevo tipo de personal (por ejemplo, técnicos de laboratorio), basta con crear una nueva clase que herede de Persona e implemente los métodos virtuales puros, sin tocar el código existente.

En contraste, la versión estructurada requería modificaciones en múltiples funciones para cada nuevo tipo de entidad, incrementando el riesgo de introducir errores y dificultando las pruebas.

Robustez y Manejo de Errores

La versión 2.1 introduce un sistema de manejo de excepciones que representa una mejora cualitativa sobre la versión estructurada, donde los errores frecuentemente provocaban terminaciones abruptas del programa. La combinación de RAII para gestión de recursos y excepciones jerárquicas para errores diferenciados crea una defensa en profundidad contra condiciones anómalas.

Meyers (2014) enfatiza que "el manejo correcto de recursos en presencia de excepciones requiere RAII" (p. 197), principio que la versión 2.1 implementa consistentemente en la clase Persistencia.

Línea de Trabajo Futuro

El análisis identifica varias áreas para desarrollo futuro: implementación de una base de datos

relacional para datos persistentes más robustos, incorporación de estructuras de datos lineales (colas de prioridad para lista de espera, pilas de pila) con semántica apropiada al dominio, y refactorización para aplicar patrones de diseño como Observer (para notificaciones) y Factory (para creación de entidades).

Referencias

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1995). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

Joyanes Aguilar, L. (2020). Fundamentos de programación: Algoritmos, estructura de datos y objetos (5.a ed.). McGraw-Hill.

Meyers, S. (2014). Effective Modern C++. O'Reilly Media.

Stroustrup, B. (2014). Programming: Principles and practice using C++ (2.a ed.). Addison-Wesley.